

Containers na nuvem

Relatório final de containerização, publicação no Docker Hub e execução em uma instância EC2.

Aluno	RM	Turma
Guilherme Silva dos Santos	551168	4ESPX

Uma entrega reproduzível

A mesma imagem Docker foi validada, publicada no Docker Hub e executada em uma EC2 pública na região us-east-1.

Item	Resultado validado
Portal	Site HTML personalizado com conteúdo sobre namespaces e cgroups
Imagem	guilhermessilva2004/cp04-site:cp04
Infraestrutura	Terraform criou 7 recursos na AWS
EC2	i-0c0129d1fa1b7f999 t2.micro running
Acesso público	http://3.85.240.135

Links da entrega

Recurso	Endereço
GitHub	https://github.com/GuilhermeSSantos2004/cp04-es-docker-aws
Docker Hub	https://hub.docker.com/r/guilhermessilva2004/cp04-site/tags
Portal na EC2	http://3.85.240.135

A infraestrutura utiliza uma VPC 10.0.0.0/24, sub-rede pública, Internet Gateway, rota 0.0.0.0/0 e Security Group liberando HTTP na porta 80. O IP público é temporário e pode mudar se a instância for recriada.

Build, container e publicação

Os comandos abaixo foram executados no AWS CloudShell durante a validação do projeto.

Imagem criada e container saudável

```
docker build -t guilhermessilva2004/cp04-site:cp04 .  
=> naming to docker.io/guilhermessilva2004/cp04-site:cp04  
docker ps  
cp04-evidence ... Up ... (healthy) 0.0.0.0:8080->80/tcp
```

Publicação no Docker Hub

```
Imagem pública: guilhermessilva2004/cp04-site:cp04  
Digest validado: sha256:79cd4905b532da7bf7d7d309a69c171914e90f4423688c6450481a8427091672
```

O PDF também acompanha as capturas de tela disponíveis na pasta docs/evidencias, organizadas por assunto e numeradas para conferência.

Namespaces e cgroups

FUNDAMENTOS

Isolar e controlar

Containers não são máquinas virtuais completas. Eles compartilham o kernel do host, mas enxergam ambientes separados e recebem limites próprios de recursos.

Namespaces

Namespaces definem **o que cada processo consegue enxergar**. Eles criam visões isoladas de processos, rede, sistema de arquivos, usuários e outros elementos do sistema operacional.

Assim, um processo dentro do container pode acreditar que possui sua própria rede e sua própria árvore de processos, mesmo compartilhando o mesmo kernel com outros containers.

PID - processos

NET - rede

MNT - arquivos

UTS - hostname

IPC - comunicação

USER - usuários

cgroups

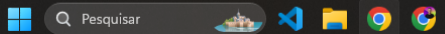
Control Groups, ou cgroups, definem **quanto cada grupo de processos pode consumir**. Eles medem, priorizam e limitam recursos do host.

Isso impede que um container monopolize a máquina e prejudique os demais serviços. Também permite acompanhar o consumo e aplicar políticas diferentes a cada aplicação.

CPU	Memória
Disco e I/O	Quantidade de processos

Juntos, namespaces e cgroups formam a base dos containers: namespaces fornecem isolamento; cgroups fornecem controle. O Docker organiza essas capacidades e adiciona imagens, redes e uma experiência padronizada de execução.

24°C
Limpo



23:43:14
28/08/2026

A página explica isolamento por namespaces e controle de recursos por cgroups. Arquivo: 01-site-namespaces-cgroups.png

Portal publicado na EC2



Captura do portal apresentado no endereço público da instância. Arquivo: 02-site-portal-ec2.png

VPC, sub-rede pública e gateway

```

CloudShell
us-east-1 +

-----
| DescribeVpcs |
|-----|
| CIDR | State | VpcId |
|-----|
| 10.0.0.0/24 | available | vpc-05cf759ee202832c5 |
|-----|

cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== SUB-REDE ==="
=== SUB-REDE ===
cp04-es-docker-aws $ aws ec2 describe-subnets --subnet-ids "$SUBNET_ID" \
> --query 'Subnets[0].{SubnetId:SubnetId,CIDR:CidrBlock,PublicIPv4:MapPublicIpOnLaunch,AZ:AvailabilityZone}' --output table

-----
| DescribeSubnets |
|-----|
| AZ | CIDR | PublicIPv4 | SubnetId |
|-----|
| us-east-1d | 10.0.0.0/26 | True | subnet-026c681c9ccff30a6 |
|-----|

cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== INTERNET GATEWAY ==="
=== INTERNET GATEWAY ===
cp04-es-docker-aws $ aws ec2 describe-internet-gateways --internet-gateway-ids "$IGW_ID" \
> --query 'InternetGateways[0].Attachments[0].{VpcId:VpcId,State:State}' --output table

-----
| DescribeInternetGateways |
|-----|
| VpcId | State |
|-----|

```

CloudShell confirma a VPC 10.0.0.0/24, a sub-rede 10.0.0.0/26 com IPv4 público automático e o Internet Gateway. Arquivo: 03-cloudshell-vpc-subnet-igw.png

Rota pública e associação

```

us-east-1 +
+-----+ DescribeRouteTables +-----+
| Destination | Gateway | State |
+-----+-----+-----+
| 10.0.0.0/24 | local | active |
| 0.0.0.0/0 | igw-02478c97747fd4a00 | active |
+-----+-----+-----+
cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== ASSOCIACAO DA SUB-REDE ==="
=== ASSOCIACAO DA SUB-REDE ===
cp04-es-docker-aws $ aws ec2 describe-route-tables --route-table-ids "$RT_ID" \
> --query 'RouteTables[0].Associations[0].SubnetId,Main:Main' --output table
+-----+ DescribeRouteTables +-----+
| Main | SubnetId |
+-----+-----+
| False | subnet-026c681c9ccff30a6 |
+-----+-----+
cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== SECURITY GROUP ==="
=== SECURITY GROUP ===
cp04-es-docker-aws $ aws ec2 describe-security-groups --group-ids "$SG_ID" \
> --query 'SecurityGroups[0].IpPermissions[0].Protocol,From:FromPort,To:ToPort,Source:join(', ', IpRanges[0].CidrIp)' --output table

```

A tabela de rotas possui 0.0.0.0/0 e está associada à sub-rede pública. Arquivo: 04-cloudshell-rotas-associacao.png

Security Group e EC2

```
us-east-1 +
cp04-es-docker-aws $ aws ec2 describe-security-groups --group-ids "sg-1u" \
> --query 'SecurityGroups[0].IpPermissions[0].{Protocol:IpProtocol,From:FromPort,To:ToPort,Source:join(", ", IpRanges[0].CidrIp)}' --output table
+-----+-----+-----+-----+
| DescribeSecurityGroups |
+-----+-----+-----+-----+
| From | Protocol | Source | To |
+-----+-----+-----+-----+
| 80 | tcp | 0.0.0.0/0 | 80 |
+-----+-----+-----+-----+
cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== EC2 ==="
=== EC2 ===
cp04-es-docker-aws $ aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
> --query 'Reservations[0].Instances[0].{InstanceId:InstanceId,State:State.Name,PublicIP:PublicIpAddress,PrivateIP:PrivateIpAddress,Type:InstanceType}' --output table
+-----+-----+-----+-----+-----+
| DescribeInstances |
+-----+-----+-----+-----+
| InstanceId | PrivateIP | PublicIP | State | Type |
+-----+-----+-----+-----+
| i-0c0129d1fa1b7f999 | 10.0.0.41 | 3.85.240.135 | running | t2.micro |
+-----+-----+-----+-----+
cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== TERRAFORM OUTPUT ==="
=== TERRAFORM OUTPUT ===
cp04-es-docker-aws $ terraform -chdir=terraform output
docker_image = "guthiermessilva2004/cp04-site:cp04"
evidence_summary = {
```

A porta TCP 80 está liberada para 0.0.0.0/0 e a consulta identifica a instância EC2 em execução. Arquivo: 05-cloudshell-security-group-ec2.png

Detalhes de rede da EC2

> i-0c0129d1fa1b7f999

<

Tipo de nome do host

Nome do IP: ip-10-0-0-41.ec2.internal

Nome do DNS do recurso privado de resposta

-

Endereço IP atribuído automaticamente

3.85.240.135 [IP público]

Função do IAM

-

IMDSv2

Required

Operador

-

Nome do DNS de IP privado (somente IPv4)

ip-10-0-0-41.ec2.internal

Tipo de instância

t2.micro

ID da VPC

vpc-05cf759ee202832c5 (cp04-es-vpc)

ID da sub-rede

subnet-026c681c9ccff30a6 (cp04-es-public-subnet)

ARN da instância

arn:aws:ec2:us-east-1:160539926514:instance/i-0c0129d1fa1b7f999

Endereços IP elásticos

-

Descoberta do AWS Compute Optimizer

Opte por participar do AWS Compute Optimizer para obter recomendações.

Saiba mais

Nome do Grupo do Auto Scaling

-

Gerenciado

falso

Detalhes

Status e alarmes

Monitoramento

Segurança

Redes

Armazenamento

Tags

Ferramentas de agentes para a AWS

Comentários

© 2026, Amazon Web Services, Inc. ou suas afiliadas.

Privacidade

Termos

Preferências de cookies

A instância mostra VPC, sub-rede, IP privado 10.0.0.41 e IP público 3.85.240.135. Arquivo: 06-ec2-detalhes-rede.png

EC2 running e Terraform

```

us-east-1 +
> --query 'Reservations[0].Instances[0].{InstanceId:InstanceId,State:State,Name:Name,PublicIP:PublicIpAddress,PrivateIP:PrivateIpAddress,Type:InstanceType}' --output table
-----
|                               DescribeInstances                               |
|-----|-----|-----|-----|-----|
| InstanceId | PrivateIP | PublicIP | State | Type |
|-----|-----|-----|-----|-----|
| i-0c0129d1fa1b7f999 | 10.0.0.41 | 3.85.240.135 | running | t2.micro |
|-----|-----|-----|-----|-----|

cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== TERRAFORM OUTPUT ==="
=== TERRAFORM OUTPUT ===
cp04-es-docker-aws $ terraform -chdir=terraform output
docker_image = "guilhermessilva2004/cp04-slte:cp04"
evidence_summary = {
  "ec2_instance_id" = "i-0c0129d1fa1b7f999"
  "ec2_public_address" = "3.85.240.135"
  "internet_gateway" = "igw-02478c97747fd4a00"
  "route_table_id" = "rtb-0330575a29e6cd653"
  "security_group_id" = "sg-0ae5c3fa7ac6d2a85"
  "subnet_id" = "subnet-026c681c9ccff30a6"
  "vpc_id" = "vpc-05cf759ee202832c5"
}
instance_id = "i-0c0129d1fa1b7f999"
public_ip = "3.85.240.135"
site_url = "http://3.85.240.135"
cp04-es-docker-aws $ Click inside the terminal window to reconnect and continue using your CloudShell session.

```

Consulta AWS confirma estado running, tipo t2.micro e o IP público da instância. Arquivo: 07-cloudshell-ec2-running.png

Saída final e site_url

```
if you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget,
commands will detect it and remind you to do so if necessary.
cp04-es-docker-aws $ terraform -chdir=terraform output
docker_image = "guilhermessilva2004/cp04-site:cp04"
evidence_summary = {
  "ec2_instance_id" = "i-0c0129d1fa1b7f999"
  "ec2_public_address" = "3.85.240.135"
  "internet_gateway" = "igw-02478c97747fd4a00"
  "route_table_id" = "rtb-0330575a29e6cd653"
  "security_group_id" = "sg-0ae5c3fa7ac6d2a85"
  "subnet_id" = "subnet-026c681c9ccff30a6"
  "vpc_id" = "vpc-05cf759ee202832c5"
}
instance_id = "i-0c0129d1fa1b7f999"
public_ip = "3.85.240.135"
site_url = "http://3.85.240.135"
cp04-es-docker-aws $
```

CloudShell Kit de ferramentas de agentes para a AWS Comentários

24°C

Saída final apresenta os IDs dos recursos e site_url = http://3.85.240.135. Arquivo: 08-cloudshell-terraform-output.png

Internet Gateway e rotas

The screenshot shows a CloudShell terminal session in the 'us-east-1' region. The user runs several AWS CLI commands to verify the configuration of an Internet Gateway and its associated Route Tables.

```

cp04-es-docker-aws $ echo "=== INTERNET GATEWAY ==="
=== INTERNET GATEWAY ===
cp04-es-docker-aws $ aws ec2 describe-internet-gateways --internet-gateway-ids "$IGW_ID" \
> --query 'InternetGateways[0].Attachments[0].{VpcId:VpcId,State:State}' --output table
+-----+-----+
| DescribeInternetGateways |
+-----+-----+
| State | VpcId |
+-----+-----+
| available | vpc-05cf759ee202832c5 |
+-----+-----+
cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== ROTAS ==="
=== ROTAS ===
cp04-es-docker-aws $ aws ec2 describe-route-tables --route-table-ids "$RT_ID" \
> --query 'RouteTables[0].Routes[0].{Destination:DestinationCidrBlock,Gateway:GatewayId,State:State}' --output table
+-----+-----+-----+
| DescribeRouteTables |
+-----+-----+-----+
| Destination | Gateway | State |
+-----+-----+-----+
| 10.0.0.0/24 | local | active |
| 0.0.0.0/0 | igw-02478c97747fd4a00 | active |
+-----+-----+-----+
cp04-es-docker-aws $
cp04-es-docker-aws $ echo "=== ASSOCIACAO DA SUB-REDE ==="
=== ASSOCIACAO DA SUB-REDE ===
cp04-es-docker-aws $ aws ec2 describe-subnet-attachments --subnet-ids "$SUB_ID" \
> --query 'SubnetAttachments[0].{VpcId:VpcId,SubnetId:SubnetId}' --output table
+-----+-----+
| DescribeSubnetAttachments |
+-----+-----+
| VpcId | SubnetId |
+-----+-----+
| vpc-05cf759ee202832c5 | subnet-0a1b2c3d4e5f6g7h |
+-----+-----+

```

The terminal output shows that the Internet Gateway is in the 'available' state and the Route Table has two routes: a local route for 10.0.0.0/24 and a default route for 0.0.0.0/0 pointing to the Internet Gateway (igw-02478c97747fd4a00) in an 'active' state.

Consulta confirma o gateway disponível e a rota padrão ativa para acesso externo. Arquivo: 09-cloudshell-igw-rotas.png

Critérios e entrega

Resumo final da implementação e dos materiais versionados no repositório.

Critério	Evidência / implementação
Site personalizado	site/index.html e capturas 01-02
Dockerfile e imagem	Dockerfile, imagem cp04 e registro textual da execução
Docker Hub	Imagem pública guilhermessilva2004/cp04-site:cp04
Rede AWS	Capturas 03-06 e configuração Terraform
EC2	Capturas 05-08; instância running e acesso HTTP público
Código e documentação	GitHub com README, checklist, evidências e este PDF

Arquivos de evidência

Todos os PNGs usados neste relatório estão em docs/evidencias/. Os nomes foram padronizados para que o professor possa localizar cada comprovação diretamente no repositório.

Observação: se a avaliação exigir prints individuais do topo do portal com nome/RM/turma, do docker build/docker ps e da página do Docker Hub, esses arquivos devem ser acrescentados à mesma pasta antes do envio final.

Conclusão

A atividade demonstra um fluxo reproduzível: o portal é empacotado em uma imagem Docker, publicado no Docker Hub e executado em uma EC2 configurada com Terraform, rede pública e acesso HTTP.